

Question 1 : Explain the fundamental differences between DDL, DML, and DQL commands in SQL. Provide one example for each type of command.

- In SQL, commands are broadly categorized based on their purpose. Three important categories are **DDL (Data Definition Language)**, **DML (Data Manipulation Language)**, and **DQL (Data Query Language)**.
-

Command Type	Purpose	Affects
DDL	Defines or modifies the database structure (schema).	Tables, databases, views, etc.
DML	Manipulates the data stored in database objects.	Records (rows) in tables
DQL	Retrieves data from the database.	Data stored in tables

- **1. DDL (Data Definition Language)**
- DDL commands are used to create, alter, or delete database objects such as tables and databases.

```
CREATE TABLE Student (  
    StudentID INT,  
    Name VARCHAR(50),  
    Age INT  
);
```

2. DML (Data Manipulation Language)

DML commands are used to insert, update, or delete data within existing tables.

```
INSERT INTO Student (StudentID, Name, Age)
```

```
VALUES (1, 'Rahul', 20);
```

3. DQL (Data Query Language)

DQL commands are used to retrieve or query data from the database. The primary DQL command is **SELECT**.

```
SELECT * FROM Student;
```

Summary

- **DDL** → Defines database structure (e.g., **CREATE**, **ALTER**, **DROP**).
- **DML** → Modifies data in tables (e.g., **INSERT**, **UPDATE**, **DELETE**).
- **DQL** → Retrieves data from tables (e.g., **SELECT**)

Q 2. What is the purpose of SQL constraints? Name and describe three common types of constraints, providing a simple scenario where each would be useful.

- **Purpose of SQL Constraints**
- **SQL constraints** are rules applied to table columns to ensure the **accuracy, consistency, and integrity of data** stored in a database. They prevent invalid data from being entered and help maintain reliable relationships between tables.

- 1. PRIMARY KEY Constraint

- **Description:**
A **PRIMARY KEY** uniquely identifies each record in a table. It does not allow duplicate values or **NULL** values.
- **Scenario:**
In a **Student** table, each student should have a unique student ID.
- **Example:**
- ```
CREATE TABLE Student (
```
- ```
    StudentID INT PRIMARY KEY,
```

- Name VARCHAR(50)
 -);
 - **Usefulness:**
Ensures that no two students have the same `StudentID`.
-

- 2. FOREIGN KEY Constraint

- **Description:**
A **FOREIGN KEY** creates a relationship between two tables and ensures that a value in one table exists in another table.
 - **Scenario:**
An `Orders` table should only contain customer IDs that already exist in the `Customers` table.
 - **Example:**
CREATE TABLE Orders (
 OrderID INT PRIMARY KEY,
 CustomerID INT,
 FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);
 - **Usefulness:**
Prevents orders from being assigned to non-existent customers.
-

- 3. NOT NULL Constraint

- **Description:**
A **NOT NULL** constraint ensures that a column cannot contain `NULL` (empty) values.
 - **Scenario:**
Every employee must have a name recorded in the database.
 - **Example:**
CREATE TABLE Employee (
 EmployeeID INT PRIMARY KEY,
 EmployeeName VARCHAR(50) NOT NULL
);
 - **Usefulness:**
Ensures that important information, such as employee names, is always provided.
-

- Summary Table

Constraint	Purpose	Example Use
PRIMARY KEY	Uniquely identifies each row	Unique Student ID
FOREIGN KEY	Maintains relationships between tables	Valid Customer ID in Orders
NOT NULL	Prevents empty values in a column	Employee Name must be entered

- These constraints help maintain **data integrity**, reduce errors, and ensure that the database stores valid and meaningful information.

Q3. Explain the difference between LIMIT and OFFSET clauses in SQL. How would you use them together to retrieve the third page of results, assuming each page has 10 records?

Difference Between **LIMIT** and **OFFSET**

Clauses	Purpose
LIMIT	Specifies the maximum number of records to return.
OFFSET	Specifies how many records to skip before starting to return records.

Example

Suppose a table contains 100 records:

```
SELECT * FROM Student  
  
LIMIT 10;
```

This returns the **first 10 records**.

```
SELECT * FROM Student  
  
LIMIT 10 OFFSET 10;
```

This skips the first 10 records and returns the **next 10 records** (records 11–20).

SQL query:

```
SELECT * FROM Student  
  
LIMIT 10 OFFSET 20;
```

Explanation:

- **LIMIT 10** → Return 10 records.
- **OFFSET 20** → Skip the first 20 records.

Q4. What is a Common Table Expression (CTE) in SQL, and what are its main benefits? Provide a simple SQL example demonstrating its usage.

Common Table Expression (CTE) in SQL

A **Common Table Expression (CTE)** is a temporary named result set that is defined using the **WITH** clause and can be referenced within a **SELECT**, **INSERT**, **UPDATE**, or **DELETE** statement.

CTEs help make complex SQL queries easier to read, write, and maintain.

Main Benefits of a CTE

1. Improves Readability

- Breaks complex queries into smaller, understandable parts.

2. Simplifies Query Maintenance

- Makes SQL code easier to modify and debug.

3. Allows Reuse of Query Results

- The result of a subquery can be referenced multiple times within the main query.

4. Supports Recursive Queries

- Useful for hierarchical data such as organizational charts or folder structures.

Syntax

```
WITH cte_name AS (  
    SELECT column1, column2  
    FROM table_name  
)  
  
SELECT *  
  
FROM cte_name;
```

Example

Suppose we have an **Employee** table and want to find employees whose salary is greater than the average salary.

```
WITH AvgSalary AS (  
    SELECT AVG(Salary) AS AverageSalary  
    FROM Employee  
)
```

```
SELECT EmployeeID, Name, Salary  
  
FROM Employee  
  
WHERE Salary > (SELECT AverageSalary FROM AvgSalary);
```

Explanation

1. The CTE `AvgSalary` calculates the average salary of all employees.
 2. The main query uses this result to select employees whose salary is above the average.
-

Without a CTE

```
SELECT EmployeeID, Name, Salary  
  
FROM Employee  
  
WHERE Salary > (  
  
    SELECT AVG(Salary)  
  
    FROM Employee  
  
);
```

While this simple example works without a CTE, CTEs become much more useful when dealing with multiple joins, nested subqueries, or recursive data structures.

Summary

- A **CTE** is a temporary named result set created using the `WITH` clause.
- It improves **readability**, **maintainability**, and **reusability** of SQL queries.
- It is especially useful for **complex queries** and **recursive operations**.

- Q5. Question 5 : Describe the concept of SQL Normalization and its primary goals. Briefly explain the first three normal forms (1NF, 2NF, 3NF).
- **SQL Normalization** is the process of organizing data in a database to reduce **data redundancy** (duplicate data) and improve **data integrity**. It involves dividing large tables into smaller, related tables and defining relationships between them.

Primary Goals of Normalization

1. **Eliminate data redundancy** – Avoid storing the same data multiple times.
2. **Improve data consistency** – Ensure accurate and reliable data.
3. **Prevent anomalies** – Avoid insertion, update, and deletion anomalies.
4. **Enhance database efficiency** – Make data management easier and more organized.

First Normal Form (1NF)

A table is in **1NF** if:

- Each column contains only **atomic (single) values**.
- There are no repeating groups or multiple values in a single column.
- Each row is unique.

Example (Not in 1NF):

Student ID	N	Subjects
101	J	MathScience

Converted to 1NF:

Student ID	N	Subject
---------------	---	---------

101	J	Math
-----	---	------

101	J	Science
-----	---	---------

Second Normal Form (2NF)

A table is in **2NF** if:

- It is already in **1NF**.
- All non-key attributes are **fully dependent on the entire primary key**, not just part of it.

Example:

Student ID	Course ID	Student Name	Course Name
---------------	--------------	-----------------	----------------

Here, **StudentName** depends only on StudentID, and **CourseName** depends only on CourseID, causing partial dependency.

Solution: Split into separate tables:

Students Table

Student ID	Student Name
---------------	-----------------

101	John
-----	------

Courses Table

Cours eID D	CourseN ame
-------------------	----------------

C1	DBMS
----	------

Enrollment Table

Stude nt ID	Cours eID D
-------------------	-------------------

Third Normal Form (3NF)

A table is in **3NF** if:

- It is already in **2NF**.
- There are **no transitive dependencies**, meaning non-key attributes should not depend on other non-key attributes.

Example:

EmployeeID	EmployeeName	DepartmentID	DepartmentName
D			

Here, **DepartmentName** depends on **DepartmentID**, not directly on **EmployeeID**.

Solution: Create separate tables:

Employee Table

EmployeeID	EmployeeName	DepartmentID
D		D

Department Table

DepartmentID	DepartmentName
D	

Summary

NormalForm	Main Requirement
1NF	Atomic values, no repeating groups
2NF	Remove partial dependencies
3NF	Remove transitive dependencies

Normalization helps create a well-structured database that minimizes redundancy and maintains data consistency.

- Q6. Question 6 : Create a database named ECommerceDB and perform the following tasks: 1. Create the following tables with appropriate data types and constraints: • Categories ◦ CategoryID (INT, PRIMARY KEY) ◦ CategoryName (VARCHAR(50), NOT NULL, UNIQUE) • Products ◦ ProductID (INT, PRIMARY KEY) ◦ ProductName (VARCHAR(100), NOT NULL, UNIQUE) ◦ CategoryID (INT, FOREIGN KEY → Categories) ◦ Price (DECIMAL(10,2), NOT NULL) ◦ StockQuantity (INT) • Customers ◦ CustomerID (INT, PRIMARY KEY) ◦ CustomerName (VARCHAR(100), NOT NULL) ◦ Email (VARCHAR(100), UNIQUE) ◦ JoinDate (DATE) • Orders ◦ OrderID (INT, PRIMARY KEY) ◦ CustomerID (INT, FOREIGN KEY → Customers) ◦ OrderDate (DATE, NOT NULL) ◦ TotalAmount (DECIMAL(10,2))
- -- Create Database
- CREATE DATABASE ECommerceDB;
-
- -- Use Database
- USE ECommerceDB;
-

```

- -- Create Categories Table
- CREATE TABLE Categories (
-     CategoryID INT PRIMARY KEY,
-     CategoryName VARCHAR(50) NOT NULL UNIQUE
- );
-
- -- Create Products Table
- CREATE TABLE Products (
-     ProductID INT PRIMARY KEY,
-     ProductName VARCHAR(100) NOT NULL UNIQUE,
-     CategoryID INT,
-     Price DECIMAL(10,2) NOT NULL,
-     StockQuantity INT,
-     FOREIGN KEY (CategoryID) REFERENCES Categories(CategoryID)
- );
-
- -- Create Customers Table
- CREATE TABLE Customers (
-     CustomerID INT PRIMARY KEY,
-     CustomerName VARCHAR(100) NOT NULL,
-     Email VARCHAR(100) UNIQUE,
-     JoinDate DATE
- );
-
- -- Create Orders Table
- CREATE TABLE Orders (
-     OrderID INT PRIMARY KEY,
-     CustomerID INT,
-     OrderDate DATE NOT NULL,
-     TotalAmount DECIMAL(10,2),
-     FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
- );

```

Q7. Question 7 : Generate a report showing CustomerName, Email, and the TotalNumberofOrders for each customer. Include customers who have not placed any orders, in which case their

TotalNumberOfOrders should be 0. Order the results by CustomerName.

```
- SELECT
-     c.CustomerName,
-     c.Email,
-     COUNT(o.OrderID) AS TotalNumberOfOrders
- FROM Customers c
- LEFT JOIN Orders o
-     ON c.CustomerID = o.CustomerID
- GROUP BY
-     c.CustomerID,
-     c.CustomerName,
-     c.Email
- ORDER BY
-     c.CustomerName;
```

Q 8. Retrieve Product Information with Category: Write a SQL query to display the ProductName, Price, StockQuantity, and CategoryName for all products. Order the results by CategoryName and then ProductName alphabetically.

```
- SELECT
-     p.ProductName,
-     p.Price,
-     p.StockQuantity,
-     c.CategoryName
- FROM Products p
- INNER JOIN Categories c
-     ON p.CategoryID = c.CategoryID
- ORDER BY
-     c.CategoryName ASC,
-     p.ProductName ASC;
```

Q9. Write a SQL query that uses a Common Table Expression (CTE) and a Window Function (specifically ROW_NUMBER() or RANK()) to display the CategoryName, ProductName, and Price for the top 2 most expensive products in each CategoryName.

```
- WITH RankedProducts AS (
```

```

- SELECT
-     c.CategoryName,
-     p.ProductName,
-     p.Price,
-     ROW_NUMBER() OVER (
-         PARTITION BY c.CategoryName
-         ORDER BY p.Price DESC
-     ) AS RowNum
- FROM Products p
- INNER JOIN Categories c
-     ON p.CategoryID = c.CategoryID
- )
-
- SELECT
-     CategoryName,
-     ProductName,
-     Price
- FROM RankedProducts
- WHERE RowNum <= 2
- ORDER BY CategoryName, Price DESC;

```

Q10. Question 10 : You are hired as a data analyst by Sakila Video Rentals, a global movie rental company. The management team is looking to improve decision-making by analyzing existing customer, rental, and inventory data. Using the Sakila database, answer the following business questions to support key strategic initiatives. Tasks & Questions: 1. Identify the top 5 customers based on the total amount they've spent. Include customer name, email, and total amount spent. 2. Which 3 movie categories have the highest rental counts? Display the category name and number of times movies from that category were rented. 3. Calculate how many films are available at each store and how many of those have never been rented. 4. Show the total revenue per month for the year 2023 to analyze business seasonality. 5. Identify customers who have rented more than 10 times in the last 6 months .

```

- SELECT
-     c.customer_id,
-     CONCAT(c.first_name, ' ', c.last_name) AS CustomerName,

```

```
-      c.email,  
-      SUM(p.amount) AS TotalAmountSpent  
- FROM customer c  
- JOIN payment p  
-      ON c.customer_id = p.customer_id  
- GROUP BY  
-      c.customer_id,  
-      c.first_name,  
-      c.last_name,  
-      c.email  
- ORDER BY TotalAmountSpent DESC  
- LIMIT 5;
```

2. SELECT

```
      cat.name AS CategoryName,  
  
      COUNT(r.rental_id) AS RentalCount  
  
FROM category cat  
  
JOIN film_category fc  
  
      ON cat.category_id = fc.category_id  
  
JOIN inventory i  
  
      ON fc.film_id = i.film_id  
  
JOIN rental r  
  
      ON i.inventory_id = r.inventory_id  
  
GROUP BY  
  
      cat.category_id,  
  
      cat.name  
  
ORDER BY RentalCount DESC
```



```
LIMIT 3;
```

```
3. SELECT
```

```
    s.store_id,
```

```
    COUNT(i.inventory_id) AS TotalFilmsAvailable,
```

```
    COUNT(CASE WHEN r.rental_id IS NULL THEN 1 END) AS  
NeverRentedFilms
```

```
FROM store s
```

```
JOIN inventory i
```

```
    ON s.store_id = i.store_id
```

```
LEFT JOIN rental r
```

```
    ON i.inventory_id = r.inventory_id
```

```
GROUP BY s.store_id;
```